

**Міністерство освіти і науки України Національний технічний університет
України «Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки Кафедра
обчислювальної техніки**

Лабораторна робота №5

з дисципліни
«Алгоритми і структури даних»

Виконала:

студентка групи ІМ-44

Крошка Дар'я Олександрівна номер у
списку групи: 11

Перевірив:

Сергієнко А. М.

Київ 2025

Постановка завдання:

Представити напрямлений граф із заданими параметрами так само, як у лабораторній роботі №3. Відмінність: коефіцієнт $k = 1.0 - 1 * 0.01 - 1 * 0.005 - 0.15$. Отже, матриця суміжності A_{dir} напрямленого графа за варіантом формується таким чином:

- 1) встановлюється параметр (seed) генератора випадкових чисел, рівне номеру варіанту $n_1 n_2 n_3 n_4$ ($n_1=4$ $n_2=4$ $n_3=1$ $n_4=1$);
- 2) матриця розміром $n * n$ заповнюється згенерованими випадковими числами в діапазоні $[0, 2.0)$;
- 3) обчислюється коефіцієнт $k = 1.0 - n_3 * 0.01 - n_4 * 0.005 - 0.15$, кожен елемент матриці множиться на коефіцієнт k ;
- 4) елементи матриці округлюються: 0 - якщо елемент менший за 1.0, 1 - якщо елемент більший або дорівнює 1.0.

2. Створити програму, яка виконує обхід напрямленого графа вшир (BFS) та вглиб (DFS).

- обхід починати з вершини із найменшим номером, яка має щонайменше одну вихідну дугу;
- при обході враховувати порядок нумерації;
- у програмі виконання обходу відображати покроково, черговий крок виконувати за натисканням кнопки у вікні або на клавіатурі.

3. Під час обходу графа побудувати дерево обходу. У програмі дерево обходу виводити покроково у процесі виконання обходу графа. Це можна виконати одним із двох способів:

- або виділяти іншим кольором ребра графа;
- або будувати дерево обходу поряд із графом.

4. Зміну статусів вершин у процесі обходу продемонструвати зміною кольорів вершин, графічними позначками тощо, або ж у процесі обходу виводити протокол обходу у графічне вікно або в консоль.

5. Якщо після обходу графа лишилися невідвідані вершини, продовжувати обхід з невідвіданої вершини з найменшим номером, яка має щонайменше одну вихідну дугу.

Код програми

```
import numpy as np
import tkinter as tk

n1, n2, n3, n4 = 4, 4, 1, 1
n = 10 + n3
seed = int(f"{n1}{n2}{n3}{n4}")
k = 1.0 - n3 * 0.01 - n4 * 0.005 - 0.15

timer_delay = 1000

np.random.seed(seed)
Adir = (np.random.rand(n, n) * 2.0 * k) >= 1.0
Adir = Adir.astype(int)
np.fill_diagonal(Adir, 0)

class GraphTraversalApp:
    def __init__(self, root, matrix):
        self.root = root
        self.matrix = matrix
        self.n = len(matrix)
        self.reset_traversal()

        self.canvas = tk.Canvas(root, width=600, height=500, bg="white")
        self.canvas.pack()

        button_frame = tk.Frame(root)
        button_frame.pack()

        self.bfs_button = tk.Button(button_frame, text="BFS Крок",
command=self.bfs_step, font=("Arial", 12))
        self.bfs_button.pack(side=tk.LEFT, padx=5)

        self.dfs_button = tk.Button(button_frame, text="DFS Крок",
command=self.dfs_step, font=("Arial", 12))
        self.dfs_button.pack(side=tk.LEFT, padx=5)

        self.reset_button = tk.Button(button_frame, text="Скинути",
command=self.reset_all, font=("Arial", 12))
        self.reset_button.pack(side=tk.LEFT, padx=5)

        self.positions = {
```

```

        i + 1: (300 + 180 * np.cos(2 * np.pi * i / self.n), 250 + 180 *
np.sin(2 * np.pi * i / self.n))
        for i in range(self.n)
    }

    self.info_label = tk.Label(root, text="", font=("Courier", 10),
justify=tk.LEFT)
    self.info_label.pack()

    self.matrix_label = tk.Label(root, text=self.format_matrix(),
font=("Courier", 8))
    self.matrix_label.pack()

    self.draw_graph()
    self.update_info()

def reset_traversal(self):
    """Reset all traversal state"""
    self.visited = set()
    self.tree_edges = []
    self.queue = []
    self.dfs_stack = []
    self.current_algorithm = None
    self.step_count = 0

    candidates = [i + 1 for i in range(self.n) if any(self.matrix[i])]
    self.start_node = min(candidates) if candidates else None

def reset_all(self):
    """Reset everything and redraw"""
    self.reset_traversal()
    self.draw_graph()
    self.update_info()

def format_matrix(self):
    """Format adjacency matrix for display"""
    header = "    " + " ".join(f"{i+1:2}" for i in range(self.n))
    rows = []
    for i in range(self.n):
        row = f"{i+1:2}: " + " ".join(f"{self.matrix[i,j]:2}" for j in
range(self.n))
        rows.append(row)
    return header + "\n" + "\n".join(rows)

```

```

def update_info(self):
    """Update information display"""
    info = f"Параметри: n1={n1}, n2={n2}, n3={n3}, n4={n4}\n"
    info += f"n={n}, seed={seed}, k={k:.3f}\n"
    info += f"Стартова вершина: {self.start_node}\n"
    info += f"Відвідані: {sorted(self.visited)}\n"
    info += f"Кроків: {self.step_count}\n"
    if self.queue:
        info += f"Чепра BFS: {self.queue}\n"
    if self.dfs_stack:
        info += f"Стек DFS: {self.dfs_stack}\n"
    info += f"Рєбра дерева: {len(self.tree_edges)}"
    self.info_label.config(text=info)

def draw_graph(self, highlight_current=None):
    """Draw the graph with current state"""
    self.canvas.delete("all")

    for i in range(1, self.n + 1):
        for j in range(1, self.n + 1):
            if self.matrix[i-1, j-1] == 1:
                x1, y1 = self.positions[i]
                x2, y2 = self.positions[j]

                if (i, j) in self.tree_edges:
                    color = "red"
                    width = 3
                else:
                    color = "gray"
                    width = 1

                self.canvas.create_line(x1, y1, x2, y2, arrow=tk.LAST,
                                         fill=color, width=width,
arrowshape=(10, 12, 3))

    for i in range(1, self.n + 1):
        x, y = self.positions[i]

        if i == highlight_current:
            color = "yellow"
        elif i in self.visited:
            color = "lightgreen"

```

```

        elif i in self.queue or i in self.dfs_stack:
            color = "lightblue"
        else:
            color = "white"

        self.canvas.create_oval(x-20, y-20, x+20, y+20,
                                fill=color, outline="black", width=2)
        self.canvas.create_text(x, y, text=str(i), font=("Arial", 12,
"bold"))

def find_next_start_node(self):
    """Find next unvisited node with outgoing edges"""
    candidates = [i + 1 for i in range(self.n)
                   if i + 1 not in self.visited and any(self.matrix[i])]
    return min(candidates) if candidates else None

def bfs_step(self):
    """Perform one step of BFS"""
    if self.current_algorithm == "DFS":
        return

    self.current_algorithm = "BFS"

    if not self.queue and not self.visited and self.start_node:
        self.queue.append(self.start_node)

    if not self.queue:
        next_start = self.find_next_start_node()
        if next_start:
            self.queue.append(next_start)
        else:
            tk.messagebox.showinfo("BFS", "BFS завершено!")
            return

    current = self.queue.pop(0)
    if current in self.visited:
        return

    self.visited.add(current)
    self.step_count += 1

    neighbors = []
    for neighbor in range(1, self.n + 1):

```

```

        if self.matrix[current-1, neighbor-1] == 1 and neighbor not in
self.visited:
            if neighbor not in self.queue:
                self.queue.append(neighbor)
                neighbors.append(neighbor)
                self.tree_edges.append((current, neighbor))

self.draw_graph(highlight_current=current)
self.update_info()

msg = f"BFS Крок {self.step_count}: Відвідали вершину {current}"
if neighbors:
    msg += f", додали до черги: {neighbors}"
print(msg)

def dfs_step(self):
    """Perform one step of DFS"""
    if self.current_algorithm == "BFS":
        return

    self.current_algorithm = "DFS"

    if not self.dfs_stack and not self.visited and self.start_node:
        self.dfs_stack.append(self.start_node)

    if not self.dfs_stack:
        next_start = self.find_next_start_node()
        if next_start:
            self.dfs_stack.append(next_start)
        else:
            tk.messagebox.showinfo("DFS", "DFS завершено!")
            return

    current = self.dfs_stack.pop()
    if current in self.visited:
        return

    self.visited.add(current)
    self.step_count += 1

    neighbors = []
    for neighbor in range(self.n, 0, -1):

```



```

        if self.matrix[current-1, neighbor-1] == 1 and neighbor not in
self.visited:
            self.dfs_stack.append(neighbor)
            neighbors.append(neighbor)
            self.tree_edges.append((current, neighbor))

    self.draw_graph(highlight_current=current)
    self.update_info()

    msg = f"DFS Крок {self.step_count}: Відвідали вершину {current}"
    if neighbors:
        msg += f", додали до стеку: {sorted(neighbors)}"
    print(msg)

if __name__ == "__main__":
    from tkinter import messagebox

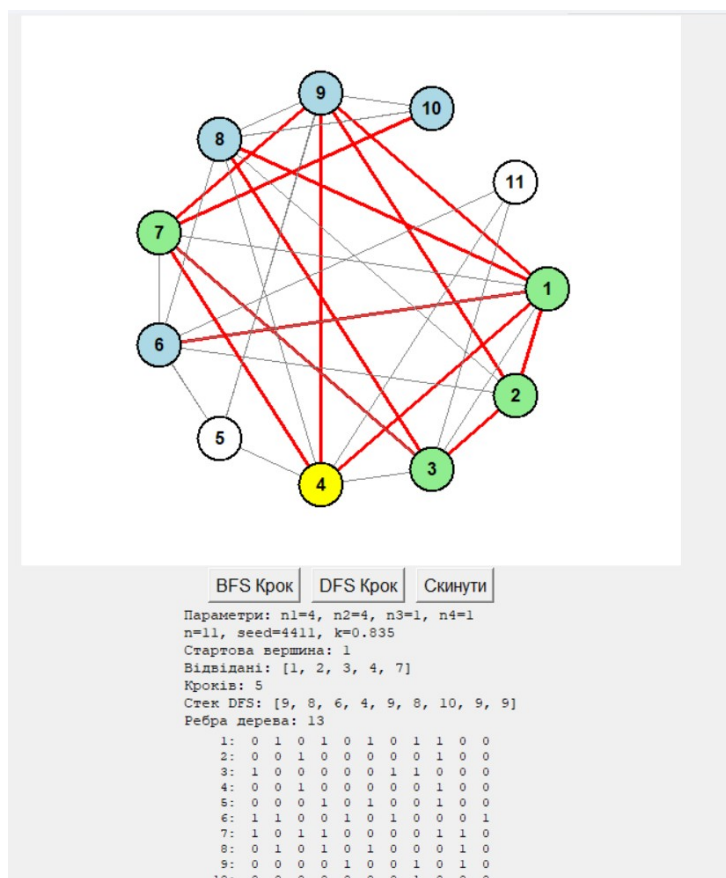
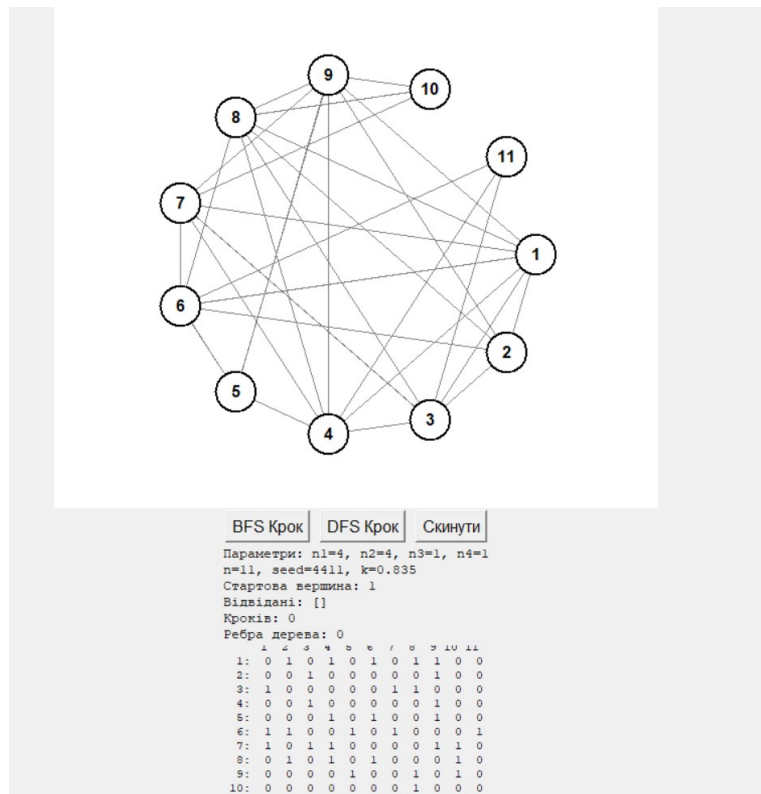
    root = tk.Tk()
    root.title("Обхід графа (BFS & DFS) - Варіант 4411")
    root.geometry("800x800")

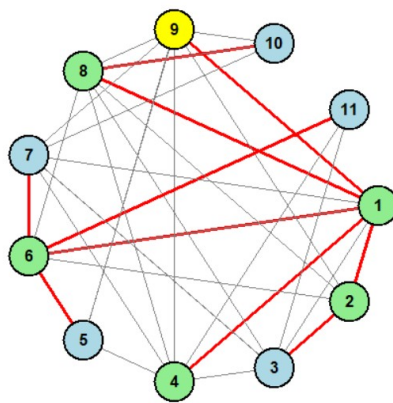
    print(f"Матриця суміжності ({n}x{n}):")
    print(Adir)
    print(f"\nПараметри: n={n}, seed={seed}, k={k:.3f}")

    app = GraphTraversalApp(root, Adir)
    root.mainloop()

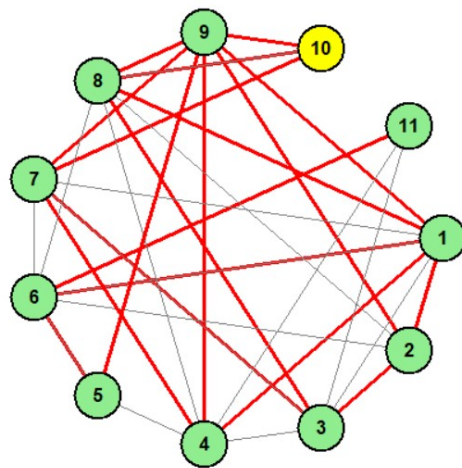
```

Скріншоти роботи програми

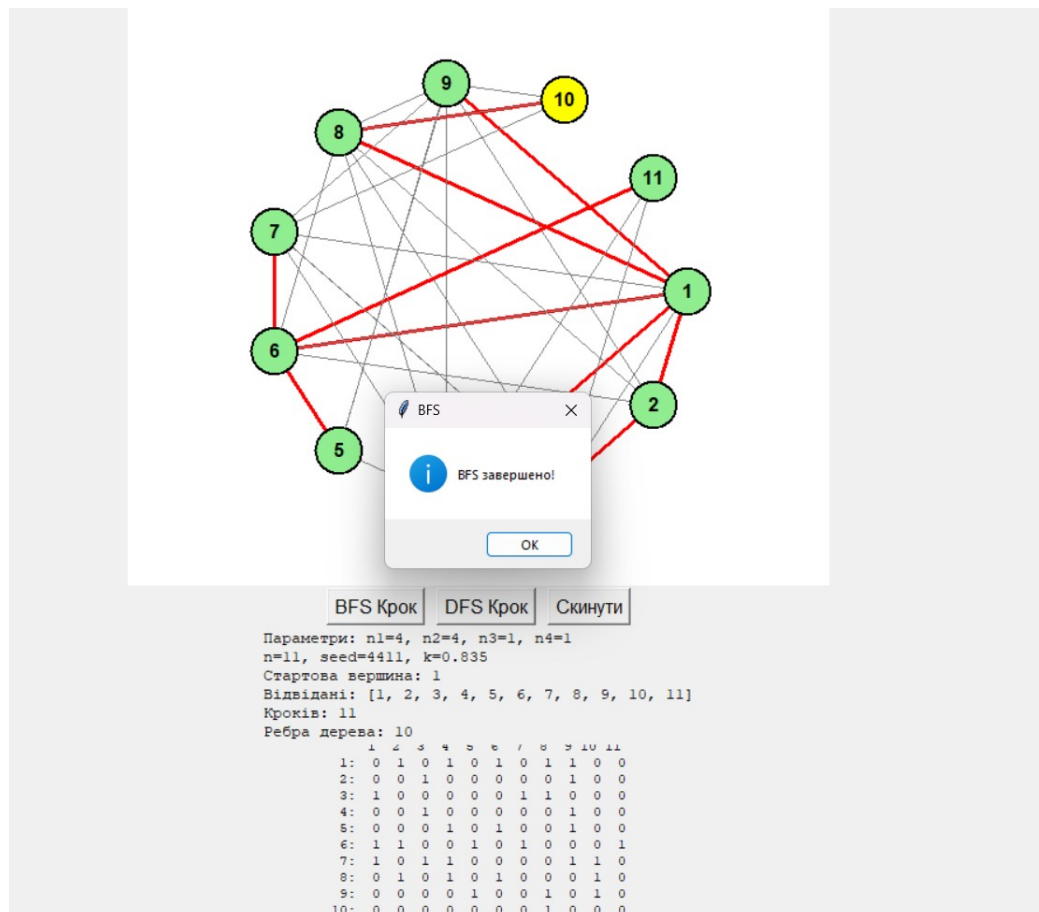




BFS Крок	DFS Крок	Скинути
Параметри: n1=4, n2=4, n3=1, n4=1		
n=11, seed=44111, k=0.835		
Стартова вершина: 1		
Відвідані: [1, 2, 4, 6, 8, 9]		
Кроків: 6		
Черга BFS: [3, 5, 7, 11, 10]		
Ребра дерева: 10		
1:	0 1 0 1 0 1 0 1 1 0 0	
2:	0 0 1 0 0 0 0 0 0 1 0 0	
3:	1 0 0 0 0 0 0 1 1 0 0 0	
4:	0 0 1 0 0 0 0 0 0 1 0 0	
5:	0 0 0 1 0 1 0 0 0 1 0 0	
6:	1 1 0 0 1 0 1 0 0 0 0 1	
7:	1 0 1 1 0 0 0 0 0 1 1 0	
8:	0 1 0 1 0 1 0 0 0 0 1 0	
9:	0 0 0 0 1 0 0 1 0 1 1 0	
10:	0 0 0 0 0 0 0 0 1 0 0 0	



BFS Крок	DFS Крок	Скинути
Параметри: n1=4, n2=4, n3=1, n4=1		
n=11, seed=44111, k=0.835		
Стартова вершина: 1		
Відвідані: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]		
Кроків: 11		
Стек DFS: [9, 8, 6, 4, 9, 8, 10, 9, 10]		
Ребра дерева: 19		
1:	0 1 0 1 0 1 0 1 1 0 0	
2:	0 0 1 0 0 0 0 0 0 1 0 0	
3:	1 0 0 0 0 0 0 1 1 0 0 0	
4:	0 0 1 0 0 0 0 0 0 1 0 0	
5:	0 0 0 1 0 1 0 0 0 1 0 0	
6:	1 1 0 0 1 0 1 0 0 0 0 1	
7:	1 0 1 1 0 0 0 0 0 1 1 0	
8:	0 1 0 1 0 1 0 0 0 0 1 0	
9:	0 0 0 0 1 0 0 1 0 1 1 0	
10:	0 0 0 0 0 0 0 0 1 0 0 0	



Матриця суміжності (11x11):

```
[
  [0 1 0 1 0 1 0 1 1 0 0],
  [0 0 1 0 0 0 0 0 1 0 0],
  [1 0 0 0 0 0 0 1 1 0 0],
  [0 0 1 0 0 0 0 0 1 0 0],
  [0 0 0 1 0 1 0 0 1 0 0],
  [1 1 0 0 1 0 1 0 0 0 1],
  [1 0 1 1 0 0 0 0 1 1 0],
  [0 1 0 1 0 1 0 0 0 1 0],
  [0 0 0 0 1 0 0 1 0 1 0],
  [0 0 0 0 0 0 0 1 0 0 0],
  [0 0 1 1 0 0 0 0 0 0 0]]
```

Параметри: $n=11$, $seed=4411$, $k=0.835$

Висновок

Розроблена програма успішно генерує та візуалізує напрямлений граф із 11 вершинами відповідно до заданих умов, використовуючи випадкову матрицю суміжності. Реалізовано покрокове виконання алгоритмів обходу в ширину (BFS) і глибину (DFS) з автоматичним вибором стартової вершини, зміною кольорів для відвіданих вершин та виділенням ребер дерева обходу. Інтерактивний графічний інтерфейс забезпечує зручне керування процесом та наочне представлення структури графа й результатів обходу.